AI LAB PROGRAMS

Python Source Code Collection — Programs 11 to 15

Includes five programs on CSP, game playing, search strategies, and decision trees.

11. Map Colouring Using CSP and Backtracking

```
# Map Colouring using CSP and Backtracking

# Regions and their neighbours
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

colours = ['Red', 'Green', 'Blue']
colouring = {}

def is_safe(region, colour):
    for neighbour in graph[region]:
        if neighbour in colouring and colouring[neighbour] == colour:
            return False
    return True

def solve(regions):
    if not regions:
        return True

    region = regions[0]

    for colour in colours:
        if is_safe(region, colour):
            colouring[region] = colour

            if solve(regions[1:]):
                return True

            del colouring[region]

    return False

if solve(list(graph.keys())):
    print("Map Colouring Solution:")
    for region, colour in colouring.items():
        print(region, ":", colour)
else:
    print("No solution exists")
```

12. Tic-Tac-Toe Game

```
# Tic-Tac-Toe Game

board = [' '] * 9

def display():
    for i in range(0, 9, 3):
        print("|".join(board[i:i+3]))
        if i < 6:
            print("-----")

def check_win(player):
    wins = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]
    return any(all(board[i] == player for i in line)
               for line in wins)

def computer_move():
    # Try to win
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            if check_win('O'):
                return
            board[i] = ' '

    # Block the human player
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            if check_win('X'):
                board[i] = 'O'
                return
            board[i] = ' '

    # Choose center, corner, or any empty cell
    for i in [4, 0, 2, 6, 8, 1, 3, 5, 7]:
        if board[i] == ' ':
            board[i] = 'O'
            return

print("Tic-Tac-Toe")
print("Positions:")
print("0 | 1 | 2")
print("3 | 4 | 5")
print("6 | 7 | 8")

while True:
    display()

    try:
        move = int(input("Enter your position (0-8): "))
    except ValueError:
        print("Enter a valid number.")
        continue

    if move < 0 or move > 8 or board[move] != ' ':
        print("Invalid move! Try again.")
        continue

    board[move] = 'X'

    if check_win('X'):
        display()
        print("You win!")
        break

    if ' ' not in board:
        display()
        print("It's a draw!")
        break

    computer_move()

    if check_win('O'):
        display()
```

```
            print("Computer wins!")
            break

    if ' ' not in board:
            display()
            print("It's a draw!")
            break
```

13. Minimax Algorithm for Tic-Tac-Toe

```
# Minimax Algorithm for Tic-Tac-Toe

board = [' '] * 9

def display():
    for i in range(0, 9, 3):
        print(" | ".join(board[i:i+3]))
        if i < 6:
            print("---------")

def check_winner(player):
    wins = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    return any(all(board[i] == player for i in line)
               for line in wins)

def minimax(is_maximizing):
    if check_winner('O'):
        return 1
    if check_winner('X'):
        return -1
    if ' ' not in board:
        return 0

    if is_maximizing:
        best_score = -float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = minimax(False)
                board[i] = ' '
                best_score = max(score, best_score)
        return best_score

    else:
        best_score = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax(True)
                board[i] = ' '
                best_score = min(score, best_score)
        return best_score

def best_move():
    best_score = -float('inf')
    move = None

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax(False)
            board[i] = ' '

            if score > best_score:
                best_score = score
                move = i

    return move

# Play the game
while True:
    display()

    try:
        move = int(input("Enter your move (0-8): "))
    except ValueError:
        print("Enter a valid number.")
        continue

    if move < 0 or move > 8 or board[move] != ' ':
        print("Invalid move!")
        continue

    board[move] = 'X'
```

```
if check_winner('X'):
    display()
    print("You win!")
    break

if ' ' not in board:
    display()
    print("Draw!")
    break

move = best_move()
board[move] = 'O'
print("Computer chooses:", move)

if check_winner('O'):
    display()
    print("Computer wins!")
    break

if ' ' not in board:
    display()
    print("Draw!")
    break
```

14. Alpha-Beta Pruning for Tic-Tac-Toe

```
# Alpha-Beta Pruning for Tic-Tac-Toe

board = [' '] * 9

def display():
    for i in range(0, 9, 3):
        print(" | ".join(board[i:i+3]))
        if i < 6:
            print("---------")

def winner(player):
    wins = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    return any(all(board[i] == player for i in line)
               for line in wins)

def alphabeta(is_max, alpha, beta):
    if winner('O'):
        return 1
    if winner('X'):
        return -1
    if ' ' not in board:
        return 0

    if is_max:
        best = -float('inf')

        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = alphabeta(False, alpha, beta)
                board[i] = ' '

                best = max(best, score)
                alpha = max(alpha, best)

                if beta <= alpha:
                    break

        return best

    else:
        best = float('inf')

        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = alphabeta(True, alpha, beta)
                board[i] = ' '

                best = min(best, score)
                beta = min(beta, best)

                if beta <= alpha:
                    break

        return best

def best_move():
    best_score = -float('inf')
    move = None

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = alphabeta(
                False, -float('inf'), float('inf')
            )
            board[i] = ' '

            if score > best_score:
                best_score = score
                move = i

    return move
```

```
# Main game
while True:
    display()

    try:
        move = int(input("Enter your move (0-8): "))
    except ValueError:
        print("Enter a valid number.")
        continue

    if move < 0 or move > 8 or board[move] != ' ':
        print("Invalid move!")
        continue

    board[move] = 'X'

    if winner('X'):
        display()
        print("You win!")
        break

    if ' ' not in board:
        display()
        print("Draw!")
        break

    move = best_move()
    board[move] = 'O'
    print("Computer chooses:", move)

    if winner('O'):
        display()
        print("Computer wins!")
        break

    if ' ' not in board:
        display()
        print("Draw!")
        break
```

15. Decision Tree Using Information Gain

```
import math
from collections import Counter

# Sample dataset
data = [
    ['Sunny', 'No'],
    ['Sunny', 'No'],
    ['Overcast', 'Yes'],
    ['Rainy', 'Yes'],
    ['Rainy', 'Yes'],
    ['Overcast', 'Yes']
]

def entropy(labels):
    total = len(labels)
    counts = Counter(labels)

    return -sum(
        (count / total) * math.log2(count / total)
        for count in counts.values()
    )

def information_gain(data, feature_index):
    labels = [row[-1] for row in data]
    total_entropy = entropy(labels)

    values = set(row[feature_index] for row in data)
    weighted_entropy = 0

    for value in values:
        subset = [row for row in data
                  if row[feature_index] == value]
        subset_labels = [row[-1] for row in subset]

        weighted_entropy += (
            len(subset) / len(data)
        ) * entropy(subset_labels)

    return total_entropy - weighted_entropy

def build_tree(data, features):
    labels = [row[-1] for row in data]

    # If all labels are the same
    if len(set(labels)) == 1:
        return labels[0]

    # If no features remain
    if not features:
        return Counter(labels).most_common(1)[0][0]

    best_feature = max(
        features,
        key=lambda f: information_gain(data, f)
    )

    tree = {best_feature: {}}

    values = set(row[best_feature] for row in data)

    for value in values:
        subset = [row for row in data
                  if row[best_feature] == value]

        remaining = [f for f in features
                     if f != best_feature]

        tree[best_feature][value] = build_tree(
            subset, remaining
        )

    return tree

# Feature index 0 represents Outlook
features = [0]
tree = build_tree(data, features)

print("Decision Tree:")
```

```
print(tree)
```